Cómo se plantea la solución

Bisección sobre la respuesta, recorrido en amplitud en una rejilla y recorrido en profundidad con dos colores

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1. Transcribir textos: bisección sobre la respuesta
2. El bosque: recorrido en amplitud sobre una rejilla
3. Dos colores: recorrido en profundidad modificado

Contenido

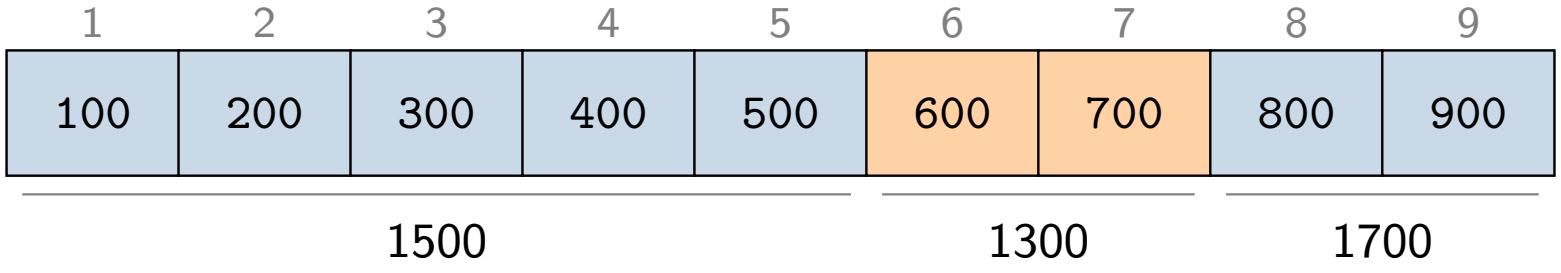
1 Transcribir textos: bisección sobre la respuesta
- El enunciado
- Qué entra y qué sale
- Por tanteo
- La idea

Transcribir textos

El problema

Hay M textos en orden, con $p_1, \ldots, p_M$ páginas, y K empleados. Cada empleado recibe textos seguidos, y cada texto queda entero con un empleado. Se tarda un minuto por página. Se pide repartir de modo que el empleado con más páginas tenga las menos posibles.

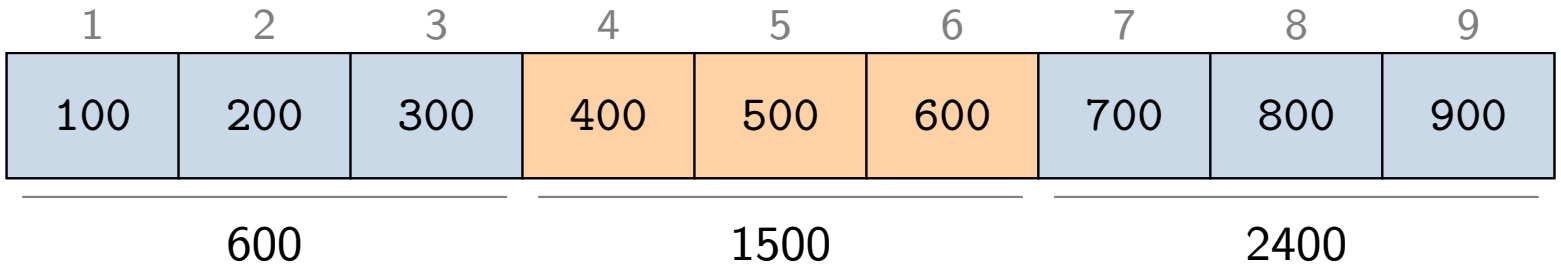
Qué entra y qué sale



Nueve textos y tres empleados

Entran la lista de páginas y $k = 3$. Sale un número, 1700: con los tramos de 1 a 5, 6 y 7, 8 y 9, los empleados llevan 1500, 1300 y 1700 páginas, y el más cargado es el que manda.

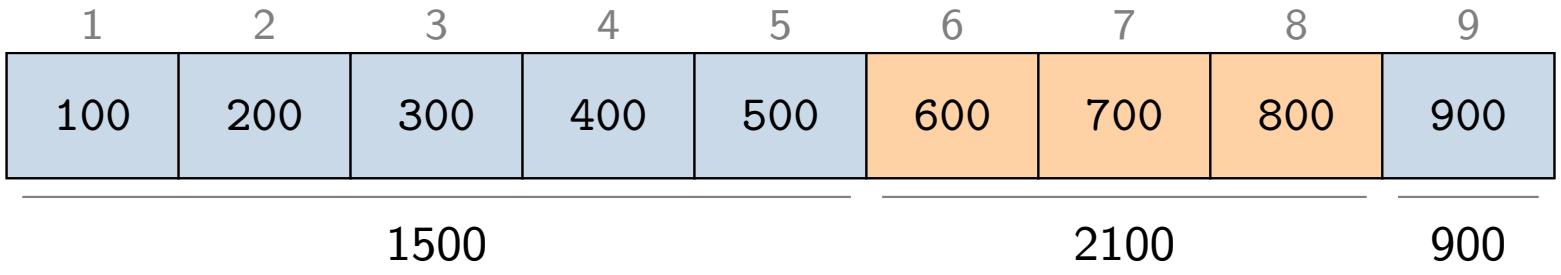
Por tanteo



Tres textos a cada uno

Con la misma cantidad de textos, el último empleado carga 2400 páginas.
Le conviene recibir menos, porque los textos del final son los más largos.

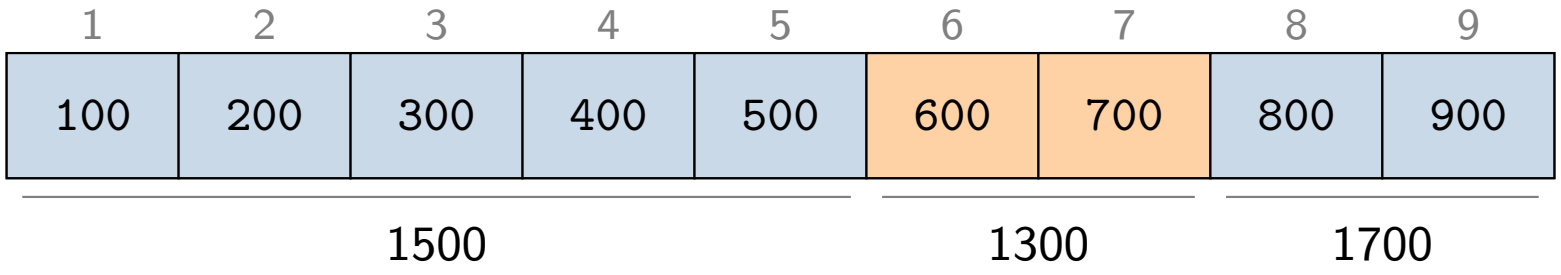
Por tanteo



Correr los cortes

El máximo baja a 2100. Ahora sobra espacio en el último tramo, y el texto 8 puede pasar allá.

Por tanteo



Otro corte más

Con el texto 8 en el último tramo, el máximo baja a 1700. ¿Se podrá con menos? Mover cortes no dice cuándo detenerse.

La forma directa: todos los repartos

M	K	Repartos
9	3	28
30	10	10 015 005
60	20	1 397 281 501 935 165

No alcanza

Repartir es escoger $K-1$ cortes entre los $M-1$ huecos, y eso da $\binom{M-1}{K-1}$ repartos. Con nueve textos son pocos. Con sesenta ya no terminan.

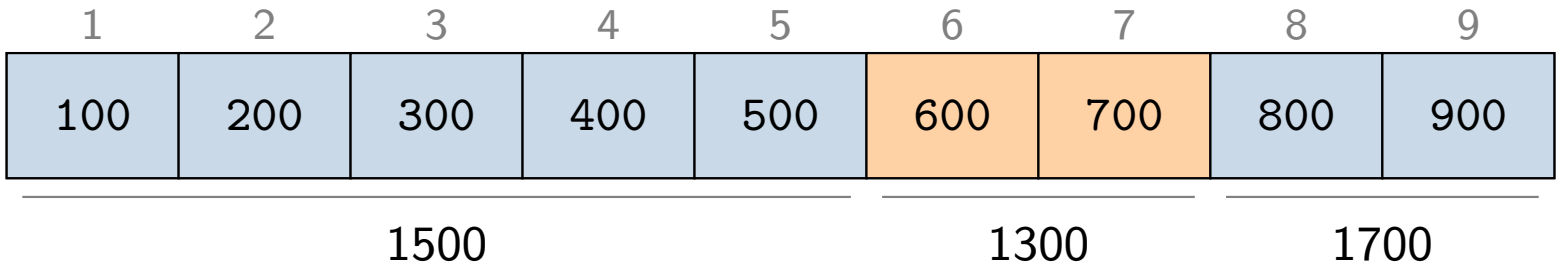
La idea: fijar un tope

1	2	3	4	5	6	7	8	9
100	200	300	400	500	600	700	800	900

Otra pregunta

Encontrar el mejor reparto es difícil. Suponga que le dan el tope: nadie puede pasar de $t = 2000$ páginas. ¿Alcanzan tres empleados?

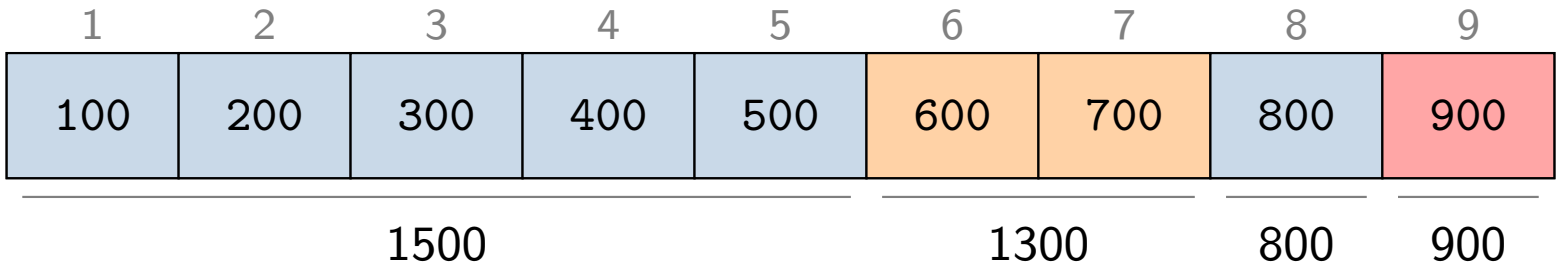
La idea: fijar un tope



Sí: con $t = 2000$ salen tres tramos

Cada empleado toma textos mientras quepan. Llenarlo no perjudica a nadie: si el primero soltara el texto 5, los demás tendrían que cargar esas 500 páginas.

La idea: fijar un tope



No: con $t = 1600$ salen cuatro tramos

El mismo llenado, una pasada por la lista, contesta para cualquier t.
Hace falta un cuarto empleado, y solo hay tres.

¿Qué le pasa a los empleados si el tope sube?

Pregunta

Con $t = 1600$ hicieron falta cuatro y con $t = 2000$ hicieron falta tres. Si t sigue subiendo, ¿puede volver a hacer falta alguien más?

Si el tope sube, los empleados no suben

t	Empleados
900	6
1200	5
1500	4
1600	4
1700	3
2000	3
2800	2
4500	1

Respuesta: no

Con un tope mayor cada empleado llega igual de lejos o más lejos, así que el siguiente arranca igual o más adelante. La columna nunca sube.

A resolver en el computador

El resultado

El programa, en Python o en C++, que lee las páginas y k e imprime la menor carga máxima: 1700 con los nueve textos.

Contenido

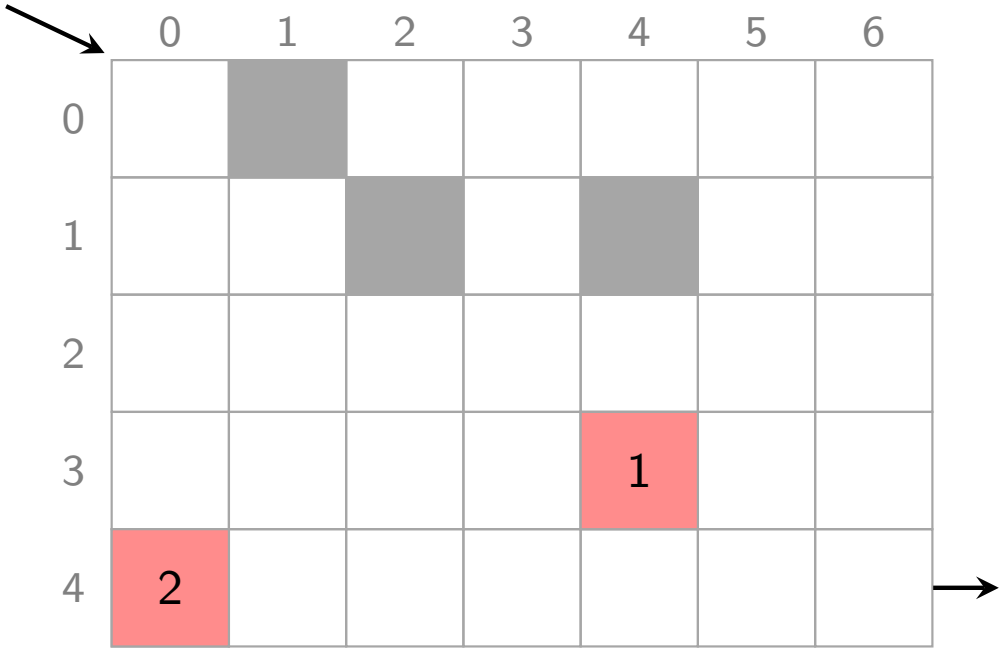
2 El bosque: recorrido en amplitud sobre una rejilla
- El enunciado
- Qué entra y qué sale
- Por tanteo
- La idea

El bosque

El problema

Un bosque es una rejilla de $R \times C$. Se entra por la esquina superior izquierda y se sale por la inferior derecha. Hay casillas bloqueadas, y en algunas vive una criatura que canta. Lo que esté a no más de `radio` unidades de ella, en cualquier dirección, es peligroso y se evita. Se pide la menor cantidad de pasos, o avisar que no se puede.

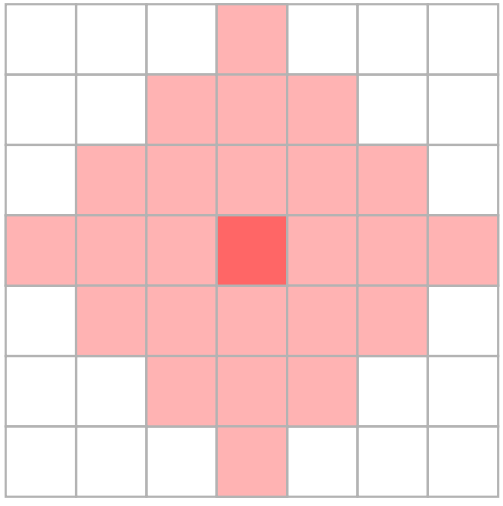
Qué entra y qué sale



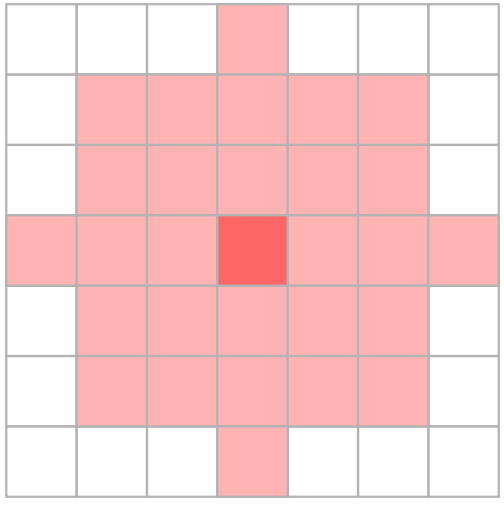
Un tablero de 5×7

Entra el tablero, con # en lo bloqueado, y una tripleta por criatura: fila y columna desde 0 y `radio`, escrito en su casilla. Aquí $(4, 0, 2)$ y $(3, 4, 1)$. Sale el número de pasos, o -1 si no se puede. Aquí, 14.

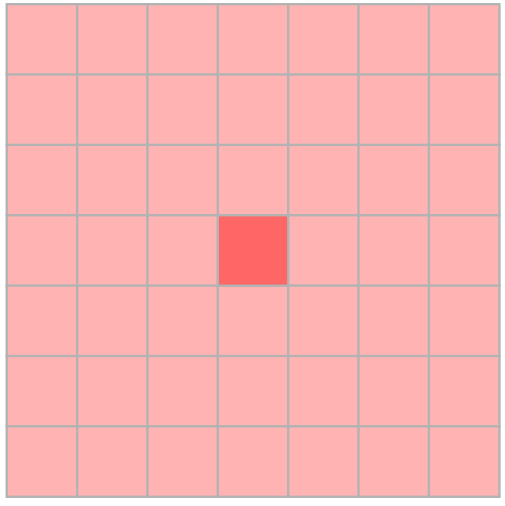
Lo que hay que decidir: “en cualquier dirección”



rombo



círculo

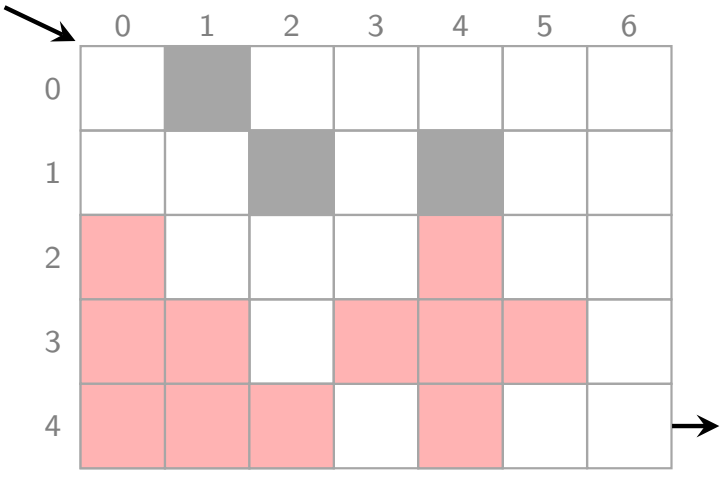


cuadrado

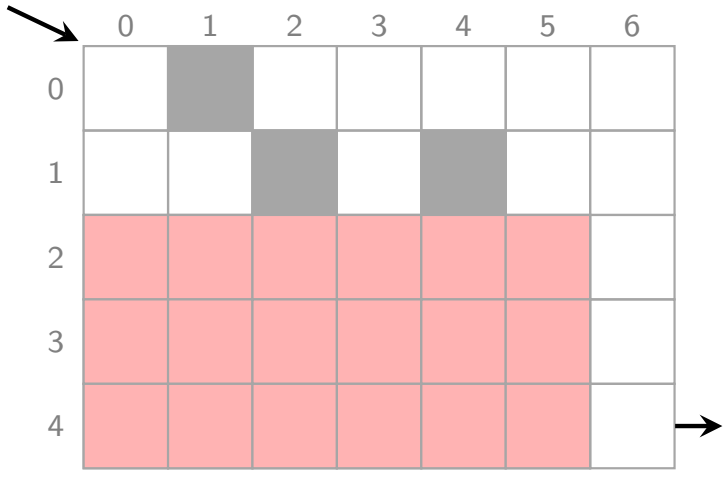
Tres lecturas con `radio` $= 3$

Contar pasos en cuatro direcciones, medir en línea recta o permitir la diagonal como un paso dan zonas distintas, con 25, 29 y 49 casillas. El texto no escoge.

El supuesto: distancia euclidiana



círculo: 14 pasos

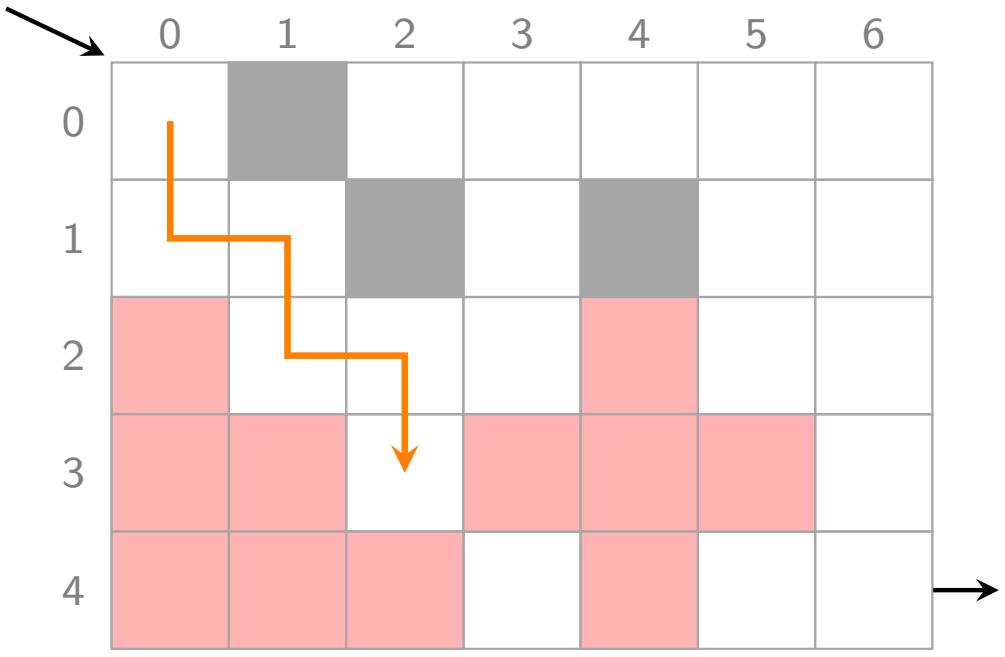


cuadrado: no se puede

Lo que se asume aquí

Se escoge el círculo: (r, c) es peligrosa si $(r - r_i)^2 + (c - c_i)^2 \leq \texttt{radio}_i^2$.
Se camina arriba, abajo, a la izquierda y a la derecha, y cada paso vale uno.

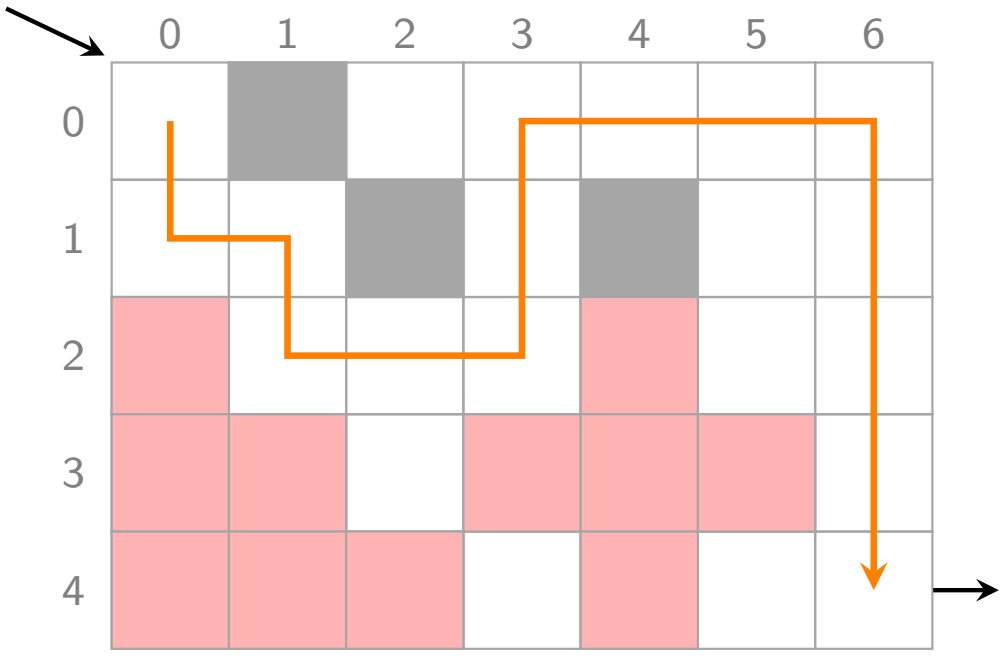
Por tanteo



Bajar por el medio

Parece lo más corto, y termina en $(3, 2)$: abajo y a los dos lados hay peligro. Hay que devolverse.

Por tanteo



Por arriba

Rodeando por arriba se llega en 14 pasos. ¿Habrá uno más corto? A ojo no se puede asegurar, y con un tablero grande ni eso.

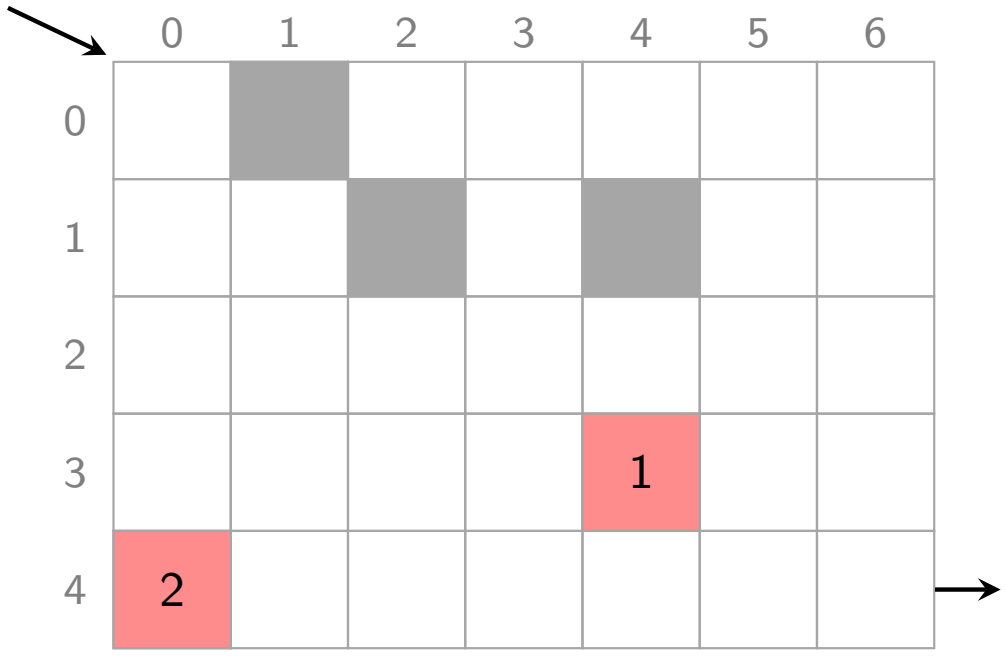
La forma directa: todos los caminos

Tablero	Caminos que solo bajan y avanzan
5×7	210
10×10	48 620
20×20	35 345 263 800

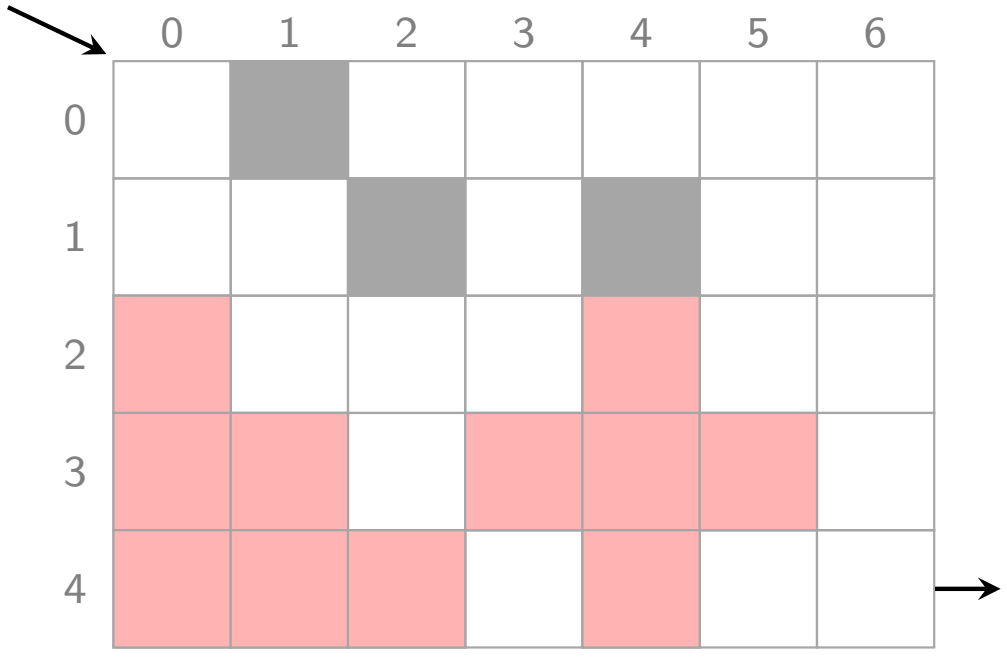
No alcanza

Esos son los caminos sin vueltas, $\binom{R+C-2}{R-1}$, y la solución puede necesitar vueltas, como en el ejemplo. Además, cada casilla pisada pregunta por las N tripletas.

¿Cambia el peligro mientras se camina?



No cambia: se marca antes de caminar



Respuesta

El peligro depende de las tripletas, nunca del camino. Una casilla peligrosa equivale a una bloqueada, y lo que queda es un tablero con paredes.

¿Y por qué no preguntar dentro del recorrido?

Pregunta

Cada casilla pisada podría revisar las N tripletas. Con $R = C = 1000$, $N = 1000$ y `radio` $= 5$, ¿cuántas consultas hace cada forma?

Marcar antes ahorra consultas

Forma	Peor caso	Con esos datos
Preguntar al pisar	$R \cdot C \cdot N$	10^9
Marcar antes	a lo sumo $N(2 \cdot \texttt{radio} + 1)^2$	121 000

Respuesta

Cada tripleta pinta una sola vez el cuadrado que la rodea. Después, saber si una casilla se puede pisar es leer una posición, y el recorrido queda igual al de siempre.

A resolver en el computador

El resultado

El programa, en Python o en C++, que lee el tablero y las tripletas e imprime los pasos, o -1: 14 con el tablero de 5×7.

Contenido

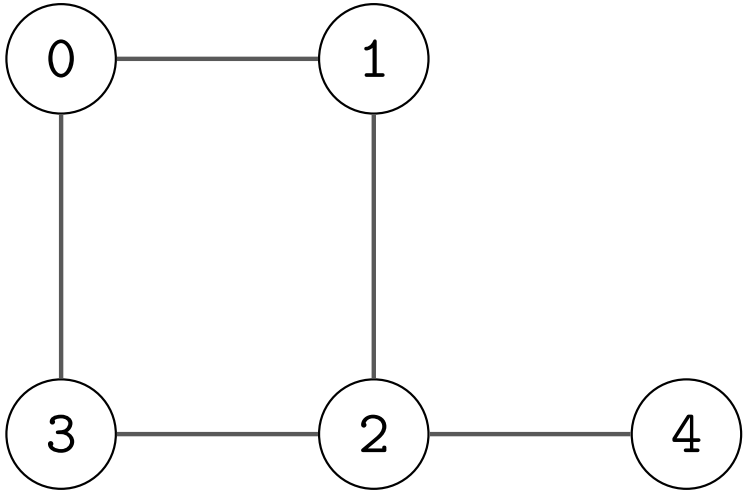
3 Dos colores: recorrido en profundidad modificado
- El enunciado
- Qué entra y qué sale
- Por tanteo
- La idea

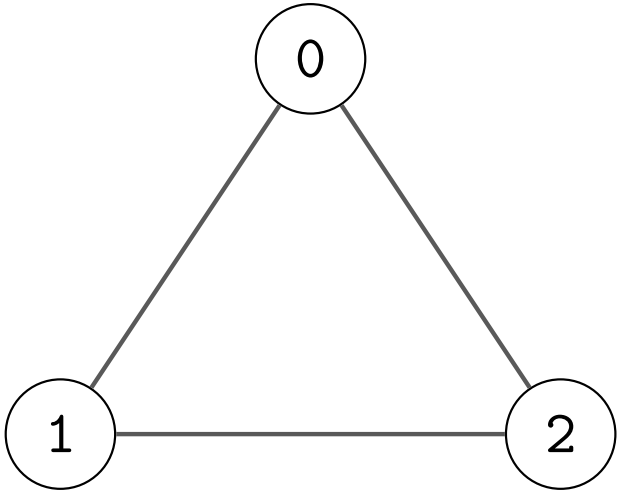
Dos colores

El problema

Se tiene un grafo no dirigido. Se quiere pintar cada nodo con uno de dos colores de modo que ninguna arista una dos nodos del mismo color. Un grafo así se llama bipartito. Se pide decidir si un grafo dado lo es.

Qué entra y qué sale

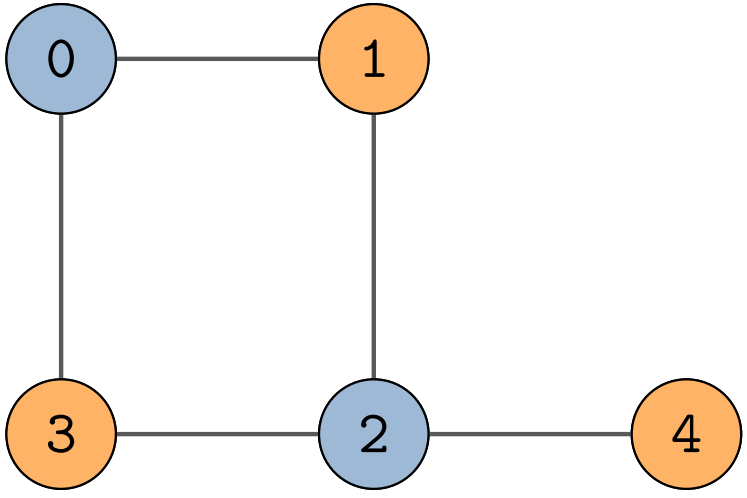


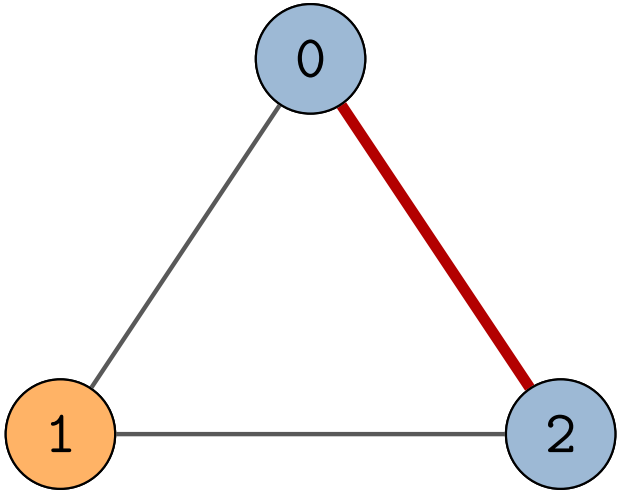


Dos grafos pequeños

Entra la lista de adyacencia. Sale cierto o falso: el de la izquierda es bipartito y el triángulo de la derecha no.

Por tanteo





Con lápiz

A la izquierda se alterna y todo encaja. A la derecha 2 no sirve en azul por 0, y tampoco en naranja por 1.

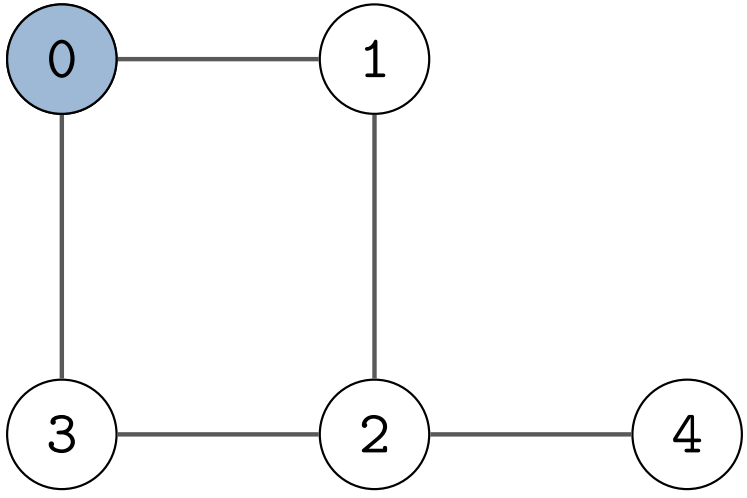
La forma directa: todos los coloreos

Nodos	Coloreos
10	1 024
30	1 073 741 824
60	1 152 921 504 606 846 976

No alcanza

Cada nodo tiene dos opciones, así que hay 2^n coloreos, y cada uno se revisa contra todas las aristas. Con sesenta nodos no termina.

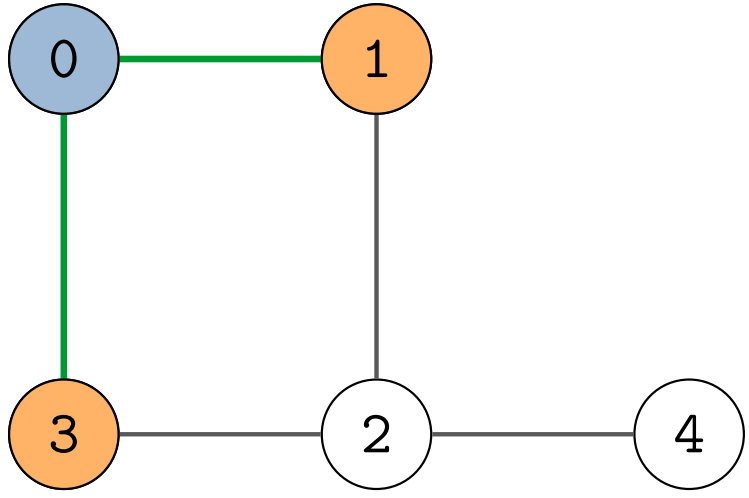
¿Hay que escoger en cada nodo?



Pregunta

El nodo 0 ya es azul. ¿De qué color pueden ser 1 y 3?

El color de un vecino no se escoge



Respuesta

Solo pueden ser naranja, y de ahí sus vecinos quedan obligados. Pintado el primer nodo, el resto de su pedazo no tiene opción: basta recorrerlo una vez.

A resolver en el computador

El resultado

El programa, en Python o en C++, que lee la lista de adyacencia y responde cierto o falso: cierto para el ciclo con la hoja, falso para el triángulo.

Referencias

- T. Cormen, C. Leiserson, R. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, 2009. Capítulo 22: recorridos en amplitud y en profundidad.
- S. Halim, F. Halim y S. Effendy. *Competitive Programming 4*: bisección sobre la respuesta.